



---

## UNIVERSITÉ DE LOMÉ

Département d'Informatique

Master en Intelligence artificielle et Big Data

INF228 — Programmation avancée 2

# Rapport de Projet du Groupe 5

## Programme Numérique pour l'Optimisation

### Par Algorithmes Génétiques

*Application au Problème du Voyageur de Commerce (TSP) — 7 Villes d'Afrique de l'Ouest*

---

#### Auteurs

NAPO NISSAO Rafia

PIO Sikirou Afeez Prince Cyrille

Année académique 2025 – 2026

Lomé, Togo

---

#### Professeurs :

Dr CHEROUAT Abel

Dr PINDRA Nadjime

Dr ABOLO-SEWOVI Komi R.

## Table des Matières

|                                                                |    |
|----------------------------------------------------------------|----|
| Table des Matières .....                                       | 2  |
| 0. Résumé Exécutif .....                                       | 3  |
| I. Compréhension du Sujet .....                                | 4  |
| 1.1. Contexte — Le Problème du Voyageur de Commerce (TSP)..... | 4  |
| 1.2. Les Algorithmes Génétiques (AG).....                      | 4  |
| 1.3. Problématique.....                                        | 4  |
| 1.4. Description des données.....                              | 5  |
| II. Implémentation Python .....                                | 6  |
| 2.1. Initialisation et représentation des données .....        | 6  |
| 2.2. Matrice des distances .....                               | 6  |
| 2.3. Encodage des solutions .....                              | 6  |
| 2.4. Population initiale et évaluation de la fitness .....     | 7  |
| 2.5. Sélection par tournoi .....                               | 7  |
| 2.6. Croisement OX (Order Crossover).....                      | 8  |
| 2.7. Mutation Swap .....                                       | 8  |
| 2.8. Boucle principale de l'algorithme génétique .....         | 8  |
| 2.9. Vérification par force brute .....                        | 9  |
| III. Résultats et Analyse.....                                 | 10 |
| 3.1. Bilan quantitatif de l'optimisation .....                 | 10 |
| 3.2. Meilleur trajet trouvé .....                              | 10 |
| 3.3. Analyse de la convergence .....                           | 10 |
| 3.4. Distribution de la population initiale .....              | 10 |
| IV. Conclusion .....                                           | 12 |
| 4.1. Bilan du projet.....                                      | 12 |
| 4.2. Apports pédagogiques .....                                | 12 |
| 4.3. Limites identifiées .....                                 | 12 |
| 4.4. Perspectives d'amélioration .....                         | 12 |
| V. Références .....                                            | 14 |

## Résumé Exécutif

Ce rapport présente l'implémentation et l'analyse d'un **algorithme génétique (AG)** appliqué au **Problème du Voyageur de Commerce (TSP)** pour 7 villes d'Afrique de l'Ouest. L'objectif est de déterminer le trajet le plus court permettant de visiter chaque ville exactement une fois avant de revenir au point de départ, en utilisant les mécanismes bio-inspirés de sélection, croisement OX et mutation swap. L'algorithme génétique implémenté en Python a convergé vers l'**optimum global** confirmé par énumération exhaustive (brute-force), réduisant la distance parcourue de 8 574 km à 5 127 km, soit une **amélioration de 40 %** par rapport à une solution aléatoire de référence.

### Paramètres clés de l'algorithme :

| Paramètre                        | Valeur                     |
|----------------------------------|----------------------------|
| Taille de la population          | 100 individus              |
| Nombre de générations            | 500                        |
| Taux de mutation                 | 10 %                       |
| Taux de croisement (OX)          | 100 %                      |
| Élitisme                         | Activé                     |
| <b>Distance optimale trouvée</b> | <b>46,1945° ≈ 5 127 km</b> |

# I. Compréhension du Sujet

## 1.1. Contexte — Le Problème du Voyageur de Commerce (TSP)

Le problème du voyageur de commerce (Travelling Salesman Problem, TSP) est l'un des problèmes combinatoires les plus étudiés en informatique théorique et en recherche opérationnelle. Il consiste à déterminer le chemin le plus court permettant de visiter un ensemble donné de villes, en passant une seule fois par chacune d'elles et en revenant au point de départ.

Les villes sont reliées par des arêtes de longueur connue, modélisées sous forme d'un graphe complet. Chaque trajet possible correspond à une permutation cyclique de l'ensemble des villes. La distance entre deux villes  $i$  et  $j$  est calculée via la formule euclidienne :

$$d(i,j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]} \quad \text{si } i \neq j, \quad d(i,j) = \infty \quad \text{si } i = j$$

L'énergie (longueur totale) d'un trajet donné par la permutation  $\sigma$  est la somme des distances entre villes successives, retour au départ inclus :

$$E = d(\sigma(1),\sigma(2)) + d(\sigma(2),\sigma(3)) + \dots + d(\sigma(N-1),\sigma(N)) + d(\sigma(N),\sigma(1))$$

## 1.2. Les Algorithmes Génétiques (AG)

Les algorithmes génétiques sont des métaheuristiques bio-inspirées, fondées sur la théorie de l'évolution darwinienne. Ils font évoluer une population de solutions candidates vers des configurations de meilleure qualité, génération après génération, par application d'opérateurs analogues à ceux de la biologie (sélection, reproduction, mutation).

Tableau 1 — Analogie biologie / algorithmique

| Concept biologique  | Équivalent algorithmique                                |
|---------------------|---------------------------------------------------------|
| Individu            | Une solution candidate (un trajet)                      |
| Chromosome          | Encodage de la solution (permutation des villes)        |
| Gène                | Un élément de la permutation (indice d'une ville)       |
| Population          | Ensemble de solutions candidates                        |
| Fitness             | Qualité d'une solution (inverse de la distance)         |
| Sélection naturelle | Favoriser les meilleures solutions pour la reproduction |
| Croisement          | Combiner deux parents pour créer un enfant              |
| Mutation            | Modifier aléatoirement une solution pour diversifier    |
| Génération          | Une itération de la boucle principale                   |

## 1.3. Problématique

**Comment peut-on utiliser un algorithme génétique pour trouver, parmi tous les trajets possibles entre N villes, celui qui parcourt la distance la plus courte en passant une seule fois par chaque ville tout en revenant au point de départ ?**

Cette question soulève immédiatement le défi de la complexité combinatoire. Le nombre de tours hamiltoniens croît de façon factorielle avec le nombre de villes, rendant toute énumération exhaustive impossible dès que N dépasse une vingtaine d'unités :

| Nombre de villes | Chemins possibles |
|------------------|-------------------|
| 7 villes         | 360 chemins       |
| 15 villes        | 43 milliards      |
| 20 villes        | 60 000 milliards  |

#### 1.4. Description des données

Sept grandes villes d'Afrique de l'Ouest ont été sélectionnées pour représenter les nœuds du graphe TSP. Ce choix présente l'avantage de ne nécessiter aucun jeu de données externe tout en offrant un ancrage géographique concret et visuellement interprétable.

Tableau 2 — Les 7 villes retenues et leurs coordonnées géographiques

| Ville       | Pays          | Longitude (°) | Latitude (°) |
|-------------|---------------|---------------|--------------|
| Lomé        | Togo          | 1,2           | 6,1          |
| Accra       | Ghana         | −0,2          | 5,6          |
| Cotonou     | Bénin         | 2,4           | 6,4          |
| Ouagadougou | Burkina Faso  | −1,5          | 12,4         |
| Bamako      | Mali          | −8,0          | 12,6         |
| Dakar       | Sénégal       | −17,4         | 14,7         |
| Abidjan     | Côte d'Ivoire | −4,0          | 5,3          |

*Note : Les distances sont calculées à partir des coordonnées géographiques exprimées en degrés. Il s'agit de distances euclidiennes approximatives et non de distances routières réelles en kilomètres.*

## II. Implémentation Python

L'algorithme a été implémenté en Python en s'appuyant sur les bibliothèques NumPy (calculs matriciels), Matplotlib et Seaborn (visualisation) ainsi que le module standard random (génération aléatoire).

### 2.1. Initialisation et représentation des données

Chaque ville est représentée comme un dictionnaire Python contenant son nom et ses coordonnées (x, y). La graine aléatoire est fixée à 42 (SEED = 42) afin d'assurer la reproductibilité des résultats.

```
import numpy as np
import matplotlib.pyplot as plt
import random, seaborn as sns

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

villes = [
    {"nom": "Lomé", "x": 1.2, "y": 6.1},
    {"nom": "Accra", "x": -0.2, "y": 5.6},
    {"nom": "Cotonou", "x": 2.4, "y": 6.4},
    {"nom": "Ouagadougou", "x": -1.5, "y": 12.4},
    {"nom": "Bamako", "x": -8.0, "y": 12.6},
    {"nom": "Dakar", "x": -17.4, "y": 14.7},
    {"nom": "Abidjan", "x": -4.0, "y": 5.3},
]
```

### 2.2. Matrice des distances

La distance euclidienne entre chaque paire de villes (i, j) est calculée selon la formule standard, puis stockée dans une matrice symétrique D de taille N×N. La diagonale est nulle (distance d'une ville à elle-même), et la matrice vérifie la propriété  $D[i][j] = D[j][i]$ .

$$d(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

```
def distance(v1, v2):
    return np.sqrt((v1["x"] - v2["x"])**2 + (v1["y"] - v2["y"])**2)

D = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        D[i][j] = distance(villes[i], villes[j])
```

### 2.3. Encodage des solutions

Une solution est représentée comme une permutation des indices de 0 à N-1. Cet encodage garantit qu'aucune ville n'est visitée deux fois et se prête directement aux opérateurs OX et swap. La longueur totale d'un trajet est calculée en additionnant les distances entre chaque paire de villes successives, en incluant le retour à la ville de départ.

```
def solution_aleatoire():
    sol = list(range(N))
    random.shuffle(sol)
    return sol
```

```
def longueur_trajet(solution):  
    total = 0  
    for i in range(N):  
        total += D[solution[i]][solution[(i + 1) % N]]  
    return total
```

## 2.4. Population initiale et évaluation de la fitness

La population initiale est composée de 100 solutions générées aléatoirement, offrant une diversité exploratoire suffisante pour que l'algorithme génétique explore efficacement l'espace des solutions. La fitness est définie comme l'inverse de la distance totale, transformant le problème de minimisation en problème de maximisation.

$$f(\text{solution}) = 1 / \text{longueur\_trajet}(\text{solution})$$

```
TAILLE_POPULATION = 100  
  
def creer_population():  
    return [solution_aleatoire() for _ in range(TAILLE_POPULATION)]  
  
def fitness(solution):  
    return 1 / longueur_trajet(solution)
```

## 2.5. Sélection par tournoi

L'opérateur de sélection retient le meilleur individu parmi un sous-ensemble de taille 5 tiré aléatoirement dans la population (sélection par tournoi). Cette approche offre un équilibre entre pression sélective et maintien de la diversité : ni trop élitiste (ce qui bloquerait la convergence prématurément), ni trop aléatoire (ce qui ralentirait la convergence).

```
TAILLE_TOURNOI = 5  
  
def selection_tournoi(population):  
    tournoi = random.sample(population, TAILLE_TOURNOI)  
    return min(tournoi, key=longueur_trajet)
```

## 2.6. Croisement OX (Order Crossover)

Le croisement classique à un ou deux points génère des doublons lorsqu'il est appliqué à des permutations. L'opérateur OX (Order Crossover) résout ce problème en conservant un sous-segment du premier parent, puis en complétant l'enfant avec les éléments manquants dans l'ordre où ils apparaissent dans le second parent.

```
def croisement_ox(parent1, parent2):
    n = len(parent1)
    i, j = sorted(random.sample(range(n), 2))

    enfant = [None] * n
    enfant[i:j] = parent1[i:j]          # segment du parent 1

    reste = [x for x in parent2 if x not in enfant]
    idx = 0
    for k in range(n):
        if enfant[k] is None:
            enfant[k] = reste[idx]
            idx += 1
    return enfant
```

## 2.7. Mutation Swap

La mutation swap intervertit aléatoirement deux villes dans une solution avec une probabilité de 10 %. Ce taux modéré permet d'éviter les minima locaux sans dégrader excessivement les bonnes solutions. L'opération préserve la validité de la permutation puisqu'elle échange deux éléments sans en supprimer ni dupliquer.

```
TAUX_MUTATION = 0.1

def mutation_swap(solution):
    sol = solution.copy()
    if random.random() < TAUX_MUTATION:
        i, j = random.sample(range(N), 2)
        sol[i], sol[j] = sol[j], sol[i]
    return sol
```

*Un taux trop élevé (> 30 %) détruirait les bonnes solutions ; un taux trop faible (< 2 %) laisserait l'algorithme bloqué dans des optima locaux. Le seuil de 10 % offre le meilleur compromis pour  $N = 7$ .*

## 2.8. Boucle principale de l'algorithme génétique

La boucle principale assemble tous les opérateurs sur 500 générations. L'élitisme est activé : la meilleure solution de chaque génération est automatiquement conservée dans la génération suivante, garantissant une évolution monotone décroissante de la distance optimale.

```
NB_GENERATIONS = 500

def algorithme_genetique():
    historique = []
    population = creer_population()
    meilleure_solution, meilleure_distance = None, float('inf')

    for gen in range(NB_GENERATIONS):
        meilleur_gen = min(population, key=longueur_trajet)
        nouvelle_pop = [meilleur_gen]          # élitisme

        while len(nouvelle_pop) < TAILLE_POPULATION:
```



```
p1 = selection_tournoi(population)
p2 = selection_tournoi(population)
enfant = croisement_ox(p1, p2)
enfant = mutation_swap(enfant)
nouvelle_pop.append(enfant)

population = nouvelle_pop
dist_gen = longueur_trajet(meilleur_gen)
if dist_gen < meilleure_distance:
    meilleure_distance = dist_gen
    meilleure_solution = meilleur_gen.copy()
    historique.append(meilleure_distance)

return meilleure_solution, meilleure_distance, historique
```

## 2.9. Vérification par force brute

Pour  $N = 7$ , il est possible d'énumérer exhaustivement les  $6! = 720$  permutations (soit 360 tours hamiltoniens distincts, par symétrie) afin de valider le résultat de l'algorithme génétique. Cette vérification constitue le test de référence (ground truth).

```
from itertools import permutations

def brute_force():
    best_dist = float('inf')
    best_sol = None
    for perm in permutations(range(1, N)):
        sol = [0] + list(perm)
        d = longueur_trajet(sol)
        if d < best_dist:
            best_dist = d
            best_sol = sol
    return best_sol, best_dist
```

## III. Résultats et Analyse

### 3.1. Bilan quantitatif de l'optimisation

L'algorithme génétique a produit les résultats suivants après 500 générations, comparés à la baseline aléatoire et à l'optimum global obtenu par force brute :

| Indicateur                                   | Valeur                                              |
|----------------------------------------------|-----------------------------------------------------|
| Baseline aléatoire (moyenne sur 200 tirages) | $\approx 8\,574$ km                                 |
| Optimum global (brute-force exhaustive)      | $46,1945^\circ \approx 5\,127$ km                   |
| <b>Distance finale trouvée par l'AG</b>      | <b><math>46,1945^\circ \approx 5\,127</math> km</b> |
| Amélioration vs. baseline                    | 40,2 %                                              |
| Écart vs. optimum global                     | $0,000000^\circ$ (optimum exact atteint)            |

L'algorithme génétique a non seulement réduit significativement la distance parcourue, mais il a également reproduit l'optimum global exact validé par l'énumération exhaustive des 360 tours hamiltoniens. Ce résultat démontre l'efficacité de l'approche bio-inspirée pour cette instance du TSP.

### 3.2. Meilleur trajet trouvé

Le trajet optimal identifié par l'algorithme génétique sur les 7 villes d'Afrique de l'Ouest est le suivant :

**Lomé → Cotonou → Ouagadougou → Bamako → Dakar → Abidjan → Accra → Lomé**

Ce trajet suit un ordre géographique cohérent, progressant vers l'ouest en longeant d'abord la côte, remontant vers le nord pour atteindre Ouagadougou et Bamako, avant de rejoindre Dakar à l'extrême ouest, puis de redescendre vers la côte guinéenne via Abidjan et Accra pour revenir à Lomé.

### 3.3. Analyse de la convergence

L'algorithme génétique converge vers l'optimum global dès les premières générations, ce qui se traduit par une courbe de convergence quasi plate sur les 500 générations. Ce comportement s'explique par la petite taille de l'instance ( $N = 7$ , soit seulement 360 solutions possibles) : l'espace de recherche est suffisamment restreint pour que la population initiale de 100 individus comprenne ou approche déjà l'optimum.

Pour des instances plus grandes ( $N > 20$ ), on observerait typiquement une phase de descente rapide dans les premières générations, suivie d'un plateau de raffinement progressif. L'élitisme garantit une évolution monotone décroissante de la meilleure distance observée.

### 3.4. Distribution de la population initiale

La population initiale présente une distribution étalée des distances, comprise entre environ 5 500 km et 9 500 km, sans concentration excessive autour d'une seule valeur. Cette diversité initiale est une condition nécessaire pour que l'algorithme génétique explore efficacement l'espace des solutions avant de converger vers l'optimum.

*Tableau 3 — Distribution des distances dans la population initiale (estimations)*

| Statistique       | Valeur approx. (km)   |
|-------------------|-----------------------|
| Distance minimale | ≈ 5 500 km            |
| Distance maximale | ≈ 9 500 km            |
| Moyenne           | ≈ 8 574 km (baseline) |

## IV. Conclusion

### 4.1. Bilan du projet

Ce projet a permis d'implémenter un algorithme génétique complet pour résoudre le Problème du Voyageur de Commerce sur 7 capitales d'Afrique de l'Ouest. Les travaux réalisés couvrent successivement :

- La modélisation du TSP avec un encodage en permutation
- L'implémentation des 4 opérateurs fondamentaux d'un AG : sélection par tournoi, croisement OX, mutation swap et élitisme
- Une réduction significative de la distance totale (de  $\approx 8\,574$  km à  $5\,127$  km, soit  $-40,2\%$ )
- La validation par comparaison avec l'optimum global obtenu par énumération exhaustive
- L'analyse de la convergence et de la distribution de la population initiale

### 4.2. Apports pédagogiques

Tableau 4 — Compétences acquises au cours de ce projet

| Domaine de compétence     | Contenu acquis                                                |
|---------------------------|---------------------------------------------------------------|
| Optimisation combinatoire | Complexité NP-difficile du TSP, espace de recherche factoriel |
| Algorithmes bio-inspirés  | Principes et opérateurs des algorithmes génétiques            |
| Encodage de solutions     | Importance du choix de la représentation (permutation)        |
| Programmation Python      | NumPy, Matplotlib, Seaborn, structures de données             |

### 4.3. Limites identifiées

Les principales limites de l'implémentation actuelle sont les suivantes :

- Pour  $N = 7$ , l'AG trouve systématiquement l'optimum exact ; pour  $N$  grand ( $> 20$ ), la solution n'est plus garantie optimale
- Les résultats peuvent légèrement varier selon l'initialisation aléatoire (sans fixation de la graine)
- Les distances utilisées sont des approximations euclidiennes et non des distances géodésiques réelles
- Les paramètres de l'algorithme (taille de population, taux de mutation) doivent être ajustés pour des instances plus grandes

### 4.4. Perspectives d'amélioration

Plusieurs pistes permettraient d'étendre et d'améliorer ce travail :

- Utiliser la formule de Haversine pour des distances géodésiques exactes entre les coordonnées GPS
- Implémenter des opérateurs de mutation plus sophistiqués : 2-opt, 3-opt, ou insertion

- Comparer les performances avec d'autres métaheuristiques : recuit simulé, colonies de fourmis (ACO)
- Étendre le problème à  $N > 7$  pour évaluer la scalabilité et ajuster les hyperparamètres
- Intégrer un mécanisme de réinitialisation partielle (restart) pour éviter la stagnation prématurée

## V. Références

---

- [1] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Davis, L. (1985). Applying adaptive algorithms to epistatic domains. *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 162–164.
- [4] NumPy Community (2023). *NumPy Documentation v1.26*. [numpy.org](https://numpy.org).
- [5] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), pp. 90–95.